EE673 Assignment-2

Question 1: Wireshark Labs

TCP

1) Client Details

Client IP Address: 192.168.1.102
Client TCP Port Number: 1161

2) Server Details

Server (gaia.cs.umass.edu) IP Address: 128.119.245.12
Server TCP Port Number: 80

4) First SYN Segment

Apply the filter `tcp.flags.syn == 1 && tcp.flags.ack == 0`.

The initial SYN sent by the client shows Seq = 0.

(The true ISN is a large random value, but Wireshark displays relative numbering starting from 0.)

What indicates that this is a SYN packet?

From the TCP header: Flags show SYN = 1.

Hence, the SYN flag bit is set.

5) SYN-ACK Segment

Apply the filter `tcp.flags.syn == 1 && tcp.flags.ack == 1`.

The server responds with Seq = 0.

(Again shown using relative numbering.)

Acknowledgment number: Ack = 1.

Ack = 1 because the client SYN had Seq = 0 and the SYN consumes one sequence number. Therefore the server sends Ack = 0 + 1 = 1.

What identifies the SYNACK?

TCP flags show SYN = 1 and ACK = 1 (both bits set).

6) HTTP POST Packet

Apply the filter `http.request.method == "POST"`.

POST /ethereal-labs/lab3-1-reply.htm HTTP/1.1

This packet has Seq = 164041.

7) RTT Calculation

Frame	Time (s)	Seq	Len
4	0.026477	1	565
5	0.041737	566	1460
7	0.054026	2026	1460
8	0.054690	3486	1460
10	0.077405	4946	1460
11	0.078157	6406	1460

Length checks:

566 - 1 = 565

2026 - 566 = 1460

3486 - 2026 = 1460

...

Segment	Sequence Number (Seq)	Length (Len)	Expected ACK (Seq + Len)
1	1	565	566
2	566	1460	2026
3	2026	1460	3486
4	3486	1460	4946
5	4946	1460	6406
6	6406	1460	7866

RTT = $ACK\ time - Send\ time$

EstimatedRTT = 0.875×Previous + 0.125×SampleRTT ($\alpha = 0.125$)

$\text{EstimatedRTT}_1 = \text{RTT}_1 = 0.053937 - 0.026477 = 27.46$ ms
$\text{RTT}_2 = 0.077294 - 0.041737 = 35.557$ ms
$\text{EstimatedRTT}_2 = 0.875 \times \text{EstimatedRTT}_1 + 0.125 \times \text{RTT}_2 = 28.47$ ms
(and similarly)
$\text{RTT}_3 = 70.06$ ms, $\text{EstimatedRTT}_3 = 33.67$ ms
$\text{RTT}_4 = 114.43$ ms, $\text{EstimatedRTT}_4 = 43.77$ ms
$\text{RTT}_5 = 139.89$ ms, $\text{EstimatedRTT}_5 = 55.78$ ms
$\text{RTT}_6 = 189.65$ ms, $\text{EstimatedRTT}_6 = 72.01$ ms

Seg	RTT (s)	EstimatedRTT (s)
1	0.027460	0.027460
2	0.035557	0.028472
3	0.070059	0.033671
4	0.114428	0.043765
5	0.139894	0.055781
6	0.189645	0.072014

8)

Segment	Frame	Sequence	Length (bytes)
1	4	1	565
2	5	566	1460
3	7	2026	1460
4	8	3486	1460

```
5          10      4946       1460
6          11      6406       1460
```

Thus, the sizes of the first six TCP segments are: 565, 1460, 1460, 1460, 1460, 1460 bytes.

9)

The smallest advertised receiver window observed in the trace is 20440 bytes.
Since the window never drops to zero and stays sufficiently large, the sender is not limited by the receiver's buffer.
Hence, receiver flow control does not constrain transmission in this trace.

10)

No retransmitted segments appear in the trace. This was confirmed using the Wireshark filter `tcp.analysis.retransmission`, which returned no matches. Furthermore, TCP sequence numbers increase strictly without repetition, verifying the absence of retransmissions.

11)

The receiver generally acknowledges about 2920 bytes at a time, corresponding to two full MSS segments of 1460 bytes each. This matches TCP delayed ACK behavior, where one cumulative ACK is sent for every two received segments. The trace repeatedly shows ACK increases of roughly 2920 bytes, confirming this pattern.

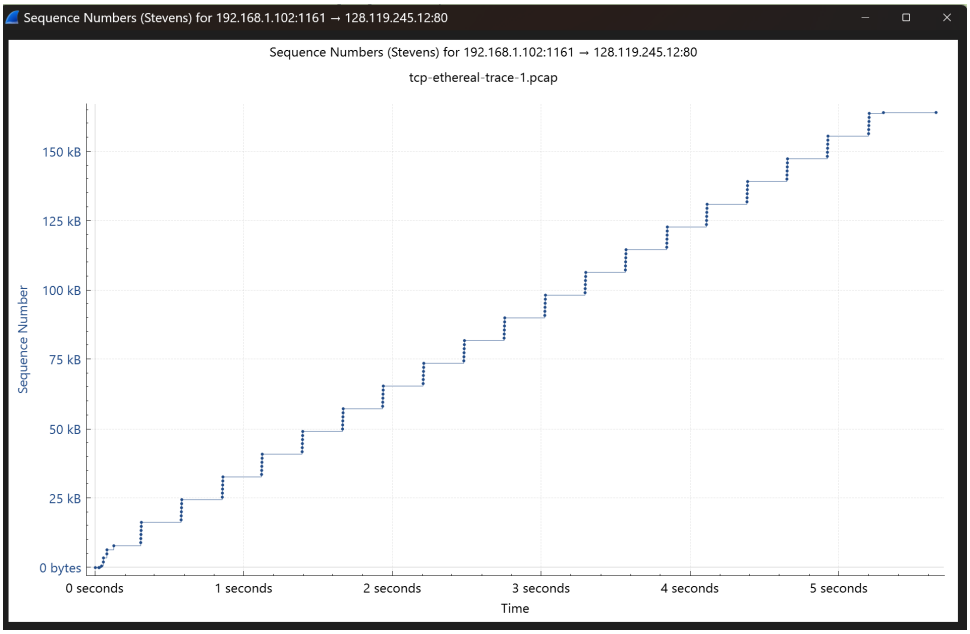
I am able to identify this in my trace by noting that after the first segment is transmitted, no immediate ACK is received, and once the second segment is sent, the acknowledgment number increases by 2920 bytes. The ACK values consistently rise in increments of +2920 rather than +1460, confirming the presence of delayed ACK behavior.

12)

The total TCP payload transferred during the connection is 164,090 bytes.
Transmission starts at time 0.026477 s and the final ACK arrives at approximately 0.216122 s, yielding a duration of 0.189645 seconds.

Throughput = Total Data Transferred / Total Time Taken
= 164,090 / 0.189645
= 865,157 bytes/sec (= 6.92 Mbps)

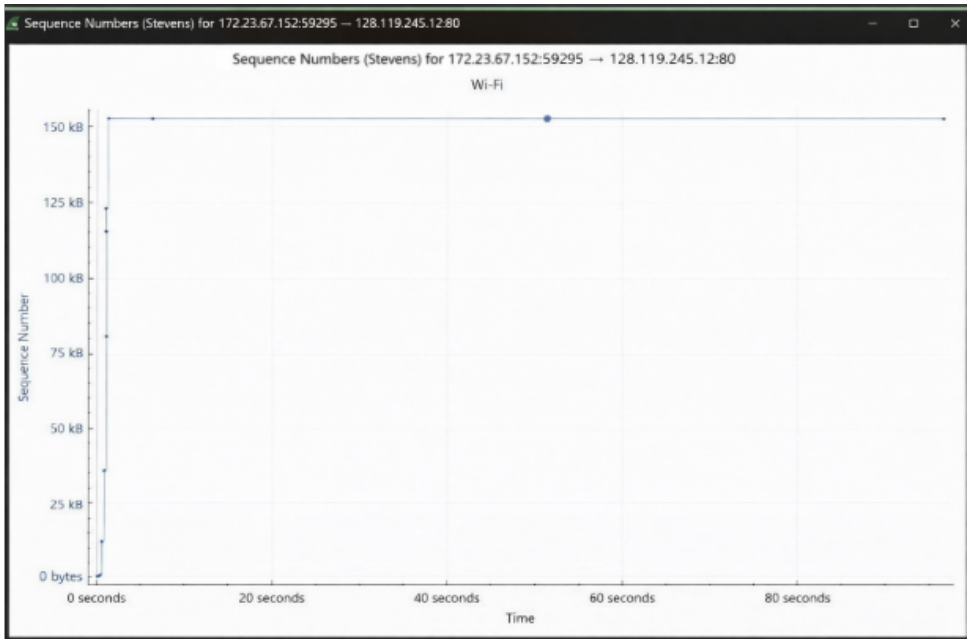
13)



The slow start phase begins immediately after the connection is established, at approximately 0 seconds. It continues until around 0.5–0.6 seconds. After this point, the congestion avoidance phase begins at approximately 0.6 seconds.

The observed behavior shows slight deviations from the ideal TCP model. During the slow start phase, the exponential increase in the congestion window does not exhibit perfect doubling, likely due to delayed acknowledgments (ACKs) and variations in network timing. Similarly, in the congestion avoidance phase, the linear growth is not perfectly uniform because of fluctuations in round-trip time (RTT) and implementation-specific factors.

14)



In the recorded trace, the slow start phase begins immediately after the connection is established. However, because the file size is relatively small and the network capacity is high, most of the data is transmitted almost instantaneously. As a result, the exponential growth characteristic of the slow start phase is very brief and not distinctly observable. Furthermore, a clear congestion avoidance phase is not visible, since the data transfer completes before the congestion window has the opportunity to increase gradually.

The throughput was calculated by dividing the total data transferred (approximately 152 kB) by the transmission time (approximately 0.29 seconds). This yields an estimated throughput of about 0.52 MB/s, which is approximately 4.2 Mbps.

UDP

1)

The UDP header contains four fields: Source Port, Destination Port, Length, and Checksum.

2)

UDP header format:
- Source Port → 2 bytes
- Destination Port → 2 bytes
- Length → 2 bytes
- Checksum → 2 bytes

Therefore, total header size = 2 + 2 + 2 + 2 = 8 bytes.

3)

The Length field specifies the complete UDP segment size (header plus data).
Length = 47
Verification: UDP header = 8 bytes
UDP data = 39 bytes
Total = 8 + 39 = 47 bytes.

4)

Length field size = 2 bytes = 16 bits
Maximum value = $2^{16} - 1 = 65535$ bytes
With an 8-byte header, the maximum UDP payload is 65535 - 8 = 65527 bytes.

5)

The source port field is 16 bits wide.
Largest possible source port number = $2^{16} - 1 = 65535$.

6)

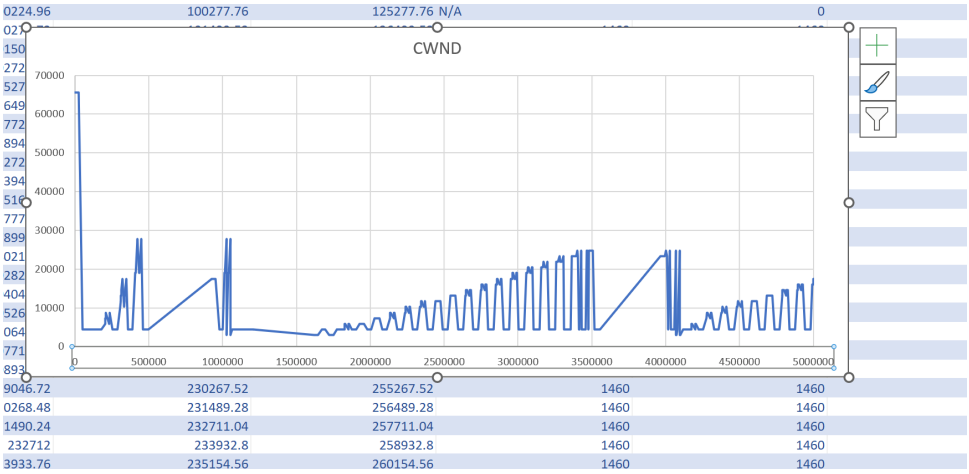
UDP protocol number:
- Decimal: 17
- Hex: 0x11

7)

First packet (DNS Query): Source Port = 58716, Destination Port = 53
Reply packet: Source Port = 53, Destination Port = 58716
In the reply, the source and destination ports are swapped.
Thus, the query's source port becomes the reply's destination port, and vice versa.

Question 2: Studying TCP Congestion Control in NetSim

Old Tahoe:

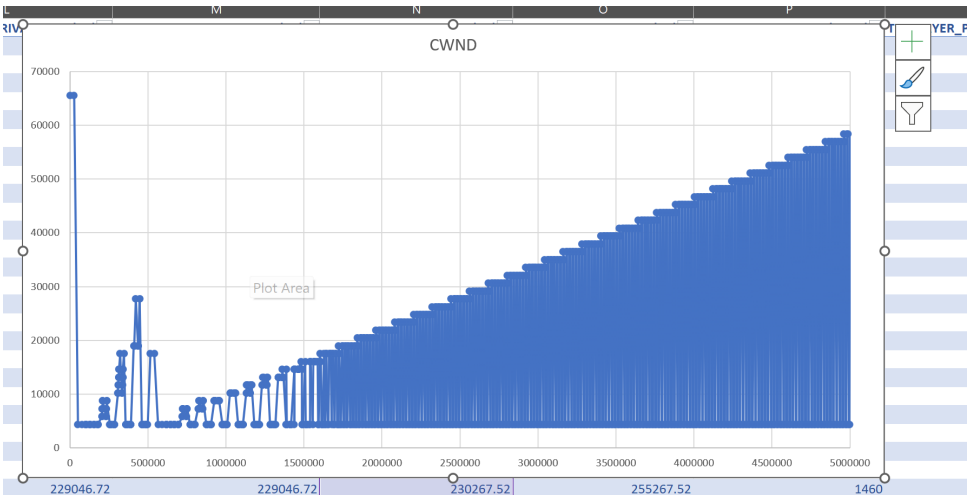


0224.96	100277.76	125277.76 N/A		0
9046.72	230267.52	255267.52	1460	1460
0268.48	231489.28	256489.28	1460	1460
1490.24	232711.04	257711.04	1460	1460
232712	233932.8	258932.8	1460	1460
3933.76	235154.56	260154.56	1460	1460

Very steep drops
Window falls to a very small value ($\sim$ 2920)
Slow start restarts frequently
Highly aggressive reset behavior

(cwnd is reduced sharply after loss, leading to slow recovery)
$\rightarrow$ Tahoe wastes bandwidth because it repeatedly re-enters slow start.

Reno:

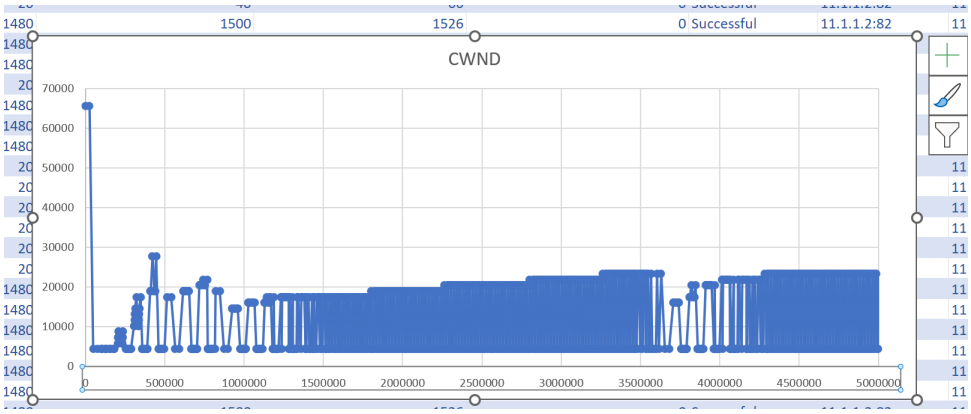


Drops are less severe than Tahoe
Minimum around 4380

Faster recovery
More stable sawtooth pattern

(uses fast retransmit with less drastic reduction)
→ Reno utilizes bandwidth more efficiently than Tahoe.

Cubic:



Growth is smoother
Achieves larger cwnd
Faster ramp-up in later phases
Minimum around 4380 (similar to Reno)

(window increases more aggressively and maintains a higher cwnd overall)
→ Cubic attains the highest average cwnd.

Algorithm	Avg CWND	Aggressiveness	Utilization
Tahoe	Low	Very conservative	Worst
Reno	Medium	Moderate	Better
Cubic	Highest	Aggressive	Best

→ For optimal link usage, the algorithm should:

- Maintain a high average cwnd
- Minimize time spent in slow start
- Recover rapidly after packet loss

→ From the cwnd vs time plots, TCP Old Tahoe shows aggressive window reduction after loss, causing frequent slow-start phases and a lower average window. TCP Reno improves performance by reducing the window less severely and recovering faster via fast retransmit and fast recovery. TCP Cubic maintains the largest steady-state congestion window and recovers fastest, thereby utilizing link capacity most efficiently. Hence, TCP Cubic achieves the highest link utilization among the three.

→ TCP Cubic utilizes the link capacity to its maximum.

Question 3: Speed Test using iPerf

- Tejasri (220941) – Server
- Sritan (220280) – Client

System Configuration:

- Server System: Windows Laptop
- Client System: Windows Laptop
- Network: iitk-sec
- Server Port: 5202
- Test Duration: 10 seconds
- Protocol Used: TCP

Test Results:

- Total Data Transferred: 41.4 MB
- Measured Bandwidth: 34.7 Mbps

Observations:

- Bandwidth varied between 19 Mbps and 49 Mbps during the test.
- Variation is due to network congestion and shared campus WiFi.
- Sender and receiver throughput values are equal, indicating stable transmission.

TCP Forward test (Client → Server):

Server Terminal

```
PS C:\Users\SRITAN\Downloads\iperf3.1.4_64> ./iperf3.exe -s -p 5202
-----------------------------------------------------------
Server listening on 5202
-----------------------------------------------------------
Accepted connection from 172.23.44.13, port 58021
[  5] local 172.23.67.152 port 5202 connected to 172.23.44.13 port 58022
[ ID] Interval           Transfer     Bandwidth
[  5]   0.00-1.00   sec  4.47 MBytes  37.5 Mbits/sec
[  5]   1.00-2.01   sec  2.65 MBytes  22.1 Mbits/sec
[  5]   2.01-3.01   sec  5.80 MBytes  48.7 Mbits/sec
[  5]   3.01-4.00   sec  3.69 MBytes  31.0 Mbits/sec
[  5]   4.00-5.00   sec  2.28 MBytes  19.1 Mbits/sec
[  5]   5.00-6.00   sec  3.81 MBytes  31.9 Mbits/sec
[  5]   6.00-7.00   sec  3.93 MBytes  33.0 Mbits/sec
[  5]   7.00-8.02   sec  5.42 MBytes  45.0 Mbits/sec
[  5]   8.02-9.01   sec  5.04 MBytes  42.7 Mbits/sec
[  5]   9.01-10.01  sec  4.20 MBytes  35.3 Mbits/sec
[  5]  10.01-10.09  sec   105 KBytes  10.0 Mbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth
[  5]   0.00-10.09  sec  0.00 Bytes  0.00 bits/sec                  sender
[  5]   0.00-10.09  sec  41.4 MBytes  34.4 Mbits/sec                  receiver
-----------------------------------------------------------
Server listening on 5202
-----------------------------------------------------------
```

Client Terminal

```
PS E:\iperf3.1.1_32> ./iperf3 -c 172.23.67.152  -p 5202
Connecting to host 172.23.67.152, port 5202
[  4] local 172.23.44.13 port 58022 connected to 172.23.67.152 port 5202
[ ID] Interval           Transfer     Bandwidth
[  4]   0.00-1.00   sec  4.61 MBytes  38.7 Mbits/sec
[  4]   1.00-2.00   sec  2.71 MBytes  22.7 Mbits/sec
[  4]   2.00-3.00   sec  5.91 MBytes  49.5 Mbits/sec
[  4]   3.00-4.00   sec  3.51 MBytes  29.4 Mbits/sec
[  4]   4.00-5.01   sec  2.28 MBytes  19.0 Mbits/sec
[  4]   5.01-6.01   sec  3.88 MBytes  32.4 Mbits/sec
[  4]   6.01-7.00   sec  4.00 MBytes  33.9 Mbits/sec
[  4]   7.00-8.00   sec  5.41 MBytes  45.3 Mbits/sec
[  4]   8.00-9.01   sec  4.98 MBytes  41.7 Mbits/sec
[  4]   9.01-10.01  sec  4.12 MBytes  34.5 Mbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth
[  4]   0.00-10.01  sec  41.4 MBytes  34.7 Mbits/sec                  sender
[  4]   0.00-10.01  sec  41.4 MBytes  34.7 Mbits/sec                  receiver

iperf Done.
```

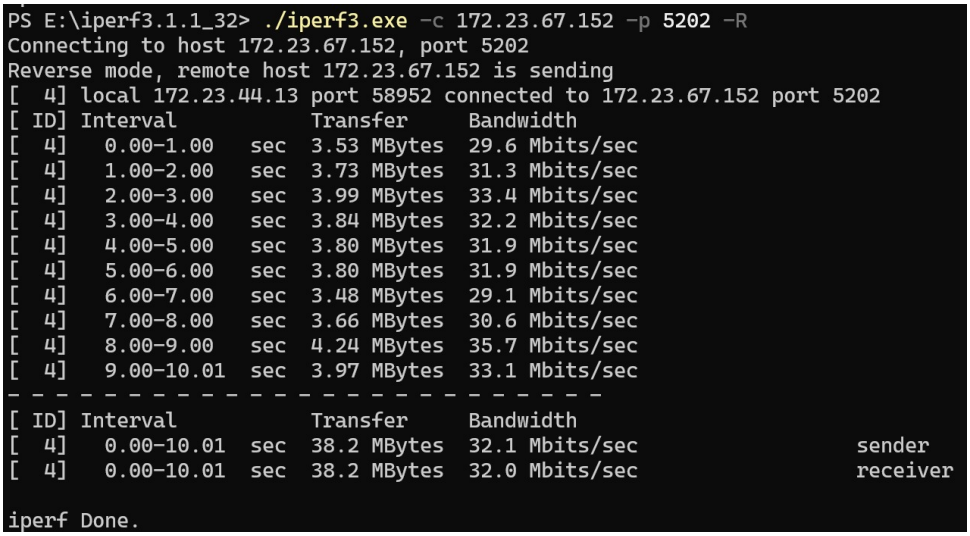
Measured bandwidth=34.7Mbps

TCP Reverse Test(Server→Client)

Server Terminal

```
-----------------------------------------------------------
Server listening on 5202
-----------------------------------------------------------
Accepted connection from 172.23.44.13, port 58951
[  5] local 172.23.67.152 port 5202 connected to 172.23.44.13 port 58952
[ ID] Interval           Transfer     Bandwidth
[  5]   0.00-1.01   sec  3.62 MBytes  30.0 Mbits/sec
[  5]   1.01-2.01   sec  3.62 MBytes  30.7 Mbits/sec
[  5]   2.01-3.01   sec  4.12 MBytes  34.5 Mbits/sec
[  5]   3.01-4.01   sec  3.75 MBytes  31.4 Mbits/sec
[  5]   4.01-5.00   sec  3.88 MBytes  32.7 Mbits/sec
[  5]   5.00-6.01   sec  3.75 MBytes  31.3 Mbits/sec
[  5]   6.01-7.01   sec  3.50 MBytes  29.2 Mbits/sec
[  5]   7.01-8.01   sec  3.62 MBytes  30.4 Mbits/sec
[  5]   8.01-9.00   sec  4.12 MBytes  35.0 Mbits/sec
[  5]   9.00-10.01  sec  4.12 MBytes  34.4 Mbits/sec
[  5]  10.01-10.05  sec   128 kBytes  27.5 Mbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth
[  5]   0.00-10.05  sec  38.2 MBytes  31.9 Mbits/sec                  sender
[  5]   0.00-10.05  sec  0.00 Bytes  0.00 bits/sec                  receiver
-----------------------------------------------------------
Server listening on 5202
-----------------------------------------------------------
```

Client Reverse



Measured Bandwidth=32.7Mbps
The reverse throughput was slightly lower due to asymmetric network conditions